

Nama : Yudhistira Putra Hartanto

Kelas/Absen : 1G/29

NIM : 254107020083

Praktikum

Percobaan 1: Operasi Penambahan pada Double Linked List

- Class Mahasiswa29
package Jobsheet12;

```
public class Mahasiswa29 {  
    String nim;  
    String nama;  
    String kelas;  
    double ipk;  
  
    public Mahasiswa29(String nim, String nama, String kelas, double ipk) {  
        this.nim = nim;  
        this.nama = nama;  
        this.kelas = kelas;  
        this.ipk = ipk;  
    }  
  
    public void tampil() {  
        System.out.println("NIM : " + nim);  
        System.out.println("Nama : " + nama);  
        System.out.println("Kelas : " + kelas);  
        System.out.println("IPK : " + ipk);  
        System.out.println();  
    }  
}
```

- Node29
package Jobsheet12;

```
public class Node29 {  
    Mahasiswa29 data;  
    Node29 prev;  
    Node29 next;  
  
    public Node29 (Mahasiswa29 data){  
        this.data = data;  
        this.prev = null;  
        this.next = null;  
    }  
}
```

- Class DoubleLinkedList29

```

package Jobsheet12;

public class DoubleLinkedList29 {
    Node29 head;
    Node29 tail;

    public DoubleLinkedList29(){
        head = null;
        tail = null;
    }

    public boolean isEmpty() {
        return head == null;
    }

    public void addFirst(Mahasiswa29 data) {
        Node29 newNode = new Node29(data);
        if (isEmpty()) {
            head = tail = newNode;
        } else {
            newNode.next = head;
            head.prev = newNode;
            head = newNode;
        }
    }

    public void addLast(Mahasiswa29 data){
        Node29 newNode = new Node29(data);
        if (isEmpty()) {
            head = tail = newNode;
        } else {
            tail.next = newNode;
            newNode.prev = tail;
            tail = newNode;
        }
    }

    public void insertAfter(String keyNim, Mahasiswa29 data) {
        Node29 current = head;
        while (current != null && !current.data.nim.equals(keyNim)) {
            current = current.next;
        }
        if (current == null) {
            System.out.println("Data dengan NIM " + keyNim + " tidak ditemukan.");
            return;
        }

        Node29 newNode = new Node29(data);

        if (current == tail) {
            newNode.prev = current;
            current.next = newNode;

```

```

        tail = newNode;
    } else {
        newNode.prev = current;
        newNode.next = current.next;
        current.next.prev = newNode;
        current.next = newNode;
    }
    System.out.println("Data berhasil disisipkan setelah NIM " + keyNim);
}

public void print() {
    if (isEmpty()) {
        System.out.println("Linked List masih kosong.");
        return;
    }

    Node29 current = head;
    while (current != null) {
        current.data.tampil();
        current = current.next;
    }
}
}

```

- Class DoubleLinkedListMain29
package Jobsheet12;

```
import java.util.Scanner;
```

```

public class DoubleLinkedListMain29 {

    //saya tambahi method ini karena nggak ada di langkah-langkah
    public static Mahasiswa29 inputMahasiswa(Scanner scan) {
        System.out.print("Masukkan NIM : ");
        String nim = scan.nextLine();
        System.out.print("Masukkan Nama : ");
        String nama = scan.nextLine();
        System.out.print("Masukkan Kelas : ");
        String kelas = scan.nextLine();
        System.out.print("Masukkan IPK : ");
        double ipk = scan.nextDouble();
        scan.nextLine();
        return new Mahasiswa29(nim, nama, kelas, ipk);
    }

    public static void main(String[] args) {
        Scanner scan = new Scanner(System.in);
        DoubleLinkedList29 list = new DoubleLinkedList29();
        int pilihan;

        do{
            System.out.println("\n===== MENU DOUBLE LINKED LIST =====");
            System.out.println("1. Tambah data di awal");
            System.out.println("2. Tambah data di akhir");

```

```

System.out.println("3. Sisipkan data di tengah (setelah NIM)");
System.out.println("4. Hapus data di awal");
System.out.println("5. Hapus data di akhir");
System.out.println("6. Tampilkan data");
System.out.println("0. Keluar");
System.out.print("Pilih menu : ");
pilihan = scan.nextInt();
scan.nextLine();

switch (pilihan) {
    case 1:
        Mahasiswa29 mhsAwal = inputMahasiswa(scan);
        list.addFirst(mhsAwal);
        break;
    case 2:
        Mahasiswa29 mhsAkhir = inputMahasiswa(scan);
        list.addLast(mhsAkhir);
        break;
    case 3:
        System.out.print("Masukkan NIM yang dicari : ");
        String keyNim = scan.nextLine();
        System.out.println("Masukkan data baru: ");
        Mahasiswa29 dataBaru = inputMahasiswa(scan);
        list.insertAfter(keyNim, dataBaru);
        break;
    case 4:
        System.out.println("Method removeFirst akan dibuat di Percobaan 2");
        break;
    case 5:
        System.out.println("Method removeLast akan dibuat di Percobaan 2");
        break;
    case 6:
        list.print();
        break;
    case 0:
        System.out.println("Program selesai.");
        break;
    default:
        System.out.println("Menu tidak valid.");
}
} while (pilihan != 0);
scan.close();
}
}

```

- Hasil
===== MENU DOUBLE LINKED LIST =====
 1. Tambah data di awal
 2. Tambah data di akhir
 3. Sisipkan data di tengah (setelah NIM)
 4. Hapus data di awal
 5. Hapus data di akhir
 6. Tampilkan data

0. Keluar
Pilih menu : 2
Masukkan NIM : 123005
Masukkan Nama : Harry
Masukkan Kelas : 1A
Masukkan IPK : 3,76

===== MENU DOUBLE LINKED LIST =====

1. Tambah data di awal
2. Tambah data di akhir
3. Sisipkan data di tengah (setelah NIM)
4. Hapus data di awal
5. Hapus data di akhir
6. Tampilkan data
0. Keluar
Pilih menu : 3
Masukkan NIM yang dicari : 123005
Masukkan data baru:
Masukkan NIM : 123010
Masukkan Nama : Potter
Masukkan Kelas : 1B
Masukkan IPK : 3,55
Data berhasil disisipkan setelah NIM 123005

===== MENU DOUBLE LINKED LIST =====

1. Tambah data di awal
2. Tambah data di akhir
3. Sisipkan data di tengah (setelah NIM)
4. Hapus data di awal
5. Hapus data di akhir
6. Tampilkan data
0. Keluar
Pilih menu : 6
NIM : 123005
Nama : Harry
Kelas : 1A
IPK : 3.76

NIM : 123010
Nama : Potter
Kelas : 1B
IPK : 3.55

===== MENU DOUBLE LINKED LIST =====

1. Tambah data di awal
2. Tambah data di akhir
3. Sisipkan data di tengah (setelah NIM)
4. Hapus data di awal
5. Hapus data di akhir
6. Tampilkan data
0. Keluar
Pilih menu : 0

Program selesai.

- **Pertanyaan**

1. Jelaskan perbedaan struktur dan mekanisme traversal antara Single Linked List dan Double Linked List!
2. Perhatikan class Node, di dalamnya terdapat atribut next dan prev. Jelaskan fungsi masing-masing atribut tersebut pada proses traversal dan manipulasi node!
3. Perhatikan konstruktor pada class DoubleLinkedList. Jelaskan fungsi konstruktor tersebut terhadap kondisi awal linked list!
4. Perhatikan potongan kode berikut:

```
if (isEmpty()) {  
    head = tail = newNode;  
}
```

Mengapa head dan tail harus menunjuk node yang sama ketika linked list masih kosong?

5. Modifikasi method print() agar menampilkan pesan "Linked List masih kosong" ketika tidak terdapat data pada linked list!
6. Modifikasi kode program dengan menambahkan method printReverse() untuk menampilkan seluruh data pada Double Linked List secara terbalik, dimulai dari node tail menuju head!

Jawaban :

1. Single Linked List itu hanya memiliki pointer next di setiap node, jadi hanya bisa maju dari head sampai tail. Sedangkan Double Linked List memiliki pointer next dan prev, jadi bisa maju mundur dari head ke tail atau tail ke head.
2. Atribut next berfungsi menyimpan referensi ke node setelahnya untuk traversal maju. Sedangkan, atribut prev berfungsi menyimpan referensi ke node sebelumnya untuk traversal mundur.
3. Konstruktor berfungsi menginisialisasi head dan tail dengan nilai null, ini itu untuk menandakan bahwa linked list masih kosong saat pertama kali dibuat.
4. Karena saat linked list hanya memiliki satu node, node tersebut sekaligus berperan sebagai node pertama (head) dan node terakhir (tail).

5.

```
public void print() {  
    if (isEmpty()) {  
        System.out.println("Linked List masih kosong.");  
        return;  
    }  
}
```

```
Node29 current = head;  
while (current != null) {  
    current.data.tampil();  
    current = current.next;  
}
```

6. kode :

```
public void printReverse() {  
    if (isEmpty()) {  
        System.out.println("Linked List masih kosong.");  
        return;  
    }  
}
```

```

Node29 current = tail;
while (current != null) {
    current.data.tampil();
    current = current.prev;
}
}

```

hasil :

===== MENU DOUBLE LINKED LIST =====

1. Tambah data di awal
2. Tambah data di akhir
3. Sisipkan data di tengah (setelah NIM)
4. Hapus data di awal
5. Hapus data di akhir
6. Tampilkan data
7. Tampilkan data secara terbalik
0. Keluar

Pilih menu : 7

NIM : 123010

Nama : Potter

Kelas : 1A

IPK : 3.55

NIM : 123005

Nama : Harry

Kelas : 1A

IPK : 3.76

Percobaan 2: Operasi Penghapusan pada Double Linked List

- Class DoubleLinkedList29

```

public void removeFirst() {
    if (isEmpty()) {
        System.out.println("Linked List kosong.");
        return;
    }
}

```

```

    if (head == tail) {
        head = tail = null;
    } else {
        head = head.next;
        head.prev = null;
    }
}

```

```

}

```

```

public void removeLast() {
    if (isEmpty()) {
        System.out.println("Linked List kosong.");
        return;
    }
}

```

```

    if (head == tail) {

```

```

        head = tail = null;
    } else {
        tail = tail.prev;
        tail.next = null;
    }
}

```

- Hasil

===== MENU DOUBLE LINKED LIST =====

1. Tambah data di awal
2. Tambah data di akhir
3. Sisipkan data di tengah (setelah NIM)
4. Hapus data di awal
5. Hapus data di akhir
6. Tampilkan data
7. Tampilkan data secara terbalik
0. Keluar

Pilih menu : 2

Masukkan NIM : 123005

Masukkan Nama : Harry

Masukkan Kelas : 1A

Masukkan IPK : 3,76

===== MENU DOUBLE LINKED LIST =====

1. Tambah data di awal
2. Tambah data di akhir
3. Sisipkan data di tengah (setelah NIM)
4. Hapus data di awal
5. Hapus data di akhir
6. Tampilkan data
7. Tampilkan data secara terbalik
0. Keluar

Pilih menu : 3

Masukkan NIM yang dicari : 123005

Masukkan data baru:

Masukkan NIM : 123010

Masukkan Nama : Potter

Masukkan Kelas : 1A

Masukkan IPK : 3,55

Data berhasil disisipkan setelah NIM 123005

===== MENU DOUBLE LINKED LIST =====

1. Tambah data di awal
2. Tambah data di akhir
3. Sisipkan data di tengah (setelah NIM)
4. Hapus data di awal
5. Hapus data di akhir
6. Tampilkan data
7. Tampilkan data secara terbalik
0. Keluar

Pilih menu : 6

NIM : 123005

Nama : Harry

Kelas : 1A
IPK : 3.76

NIM : 123010
Nama : Potter
Kelas : 1A
IPK : 3.55

===== MENU DOUBLE LINKED LIST =====

1. Tambah data di awal
 2. Tambah data di akhir
 3. Sisipkan data di tengah (setelah NIM)
 4. Hapus data di awal
 5. Hapus data di akhir
 6. Tampilkan data
 7. Tampilkan data secara terbalik
 0. Keluar
- Pilih menu : 4

===== MENU DOUBLE LINKED LIST =====

1. Tambah data di awal
 2. Tambah data di akhir
 3. Sisipkan data di tengah (setelah NIM)
 4. Hapus data di awal
 5. Hapus data di akhir
 6. Tampilkan data
 7. Tampilkan data secara terbalik
 0. Keluar
- Pilih menu : 6
NIM : 123010
Nama : Potter
Kelas : 1A
IPK : 3.55

===== MENU DOUBLE LINKED LIST =====

1. Tambah data di awal
 2. Tambah data di akhir
 3. Sisipkan data di tengah (setelah NIM)
 4. Hapus data di awal
 5. Hapus data di akhir
 6. Tampilkan data
 7. Tampilkan data secara terbalik
 0. Keluar
- Pilih menu : 1
Masukkan NIM : 123005
Masukkan Nama : Harry
Masukkan Kelas : 1A
Masukkan IPK : 3,76

===== MENU DOUBLE LINKED LIST =====

1. Tambah data di awal

2. Tambah data di akhir
 3. Sisipkan data di tengah (setelah NIM)
 4. Hapus data di awal
 5. Hapus data di akhir
 6. Tampilkan data
 7. Tampilkan data secara terbalik
 0. Keluar
- Pilih menu : 5

===== MENU DOUBLE LINKED LIST =====

1. Tambah data di awal
 2. Tambah data di akhir
 3. Sisipkan data di tengah (setelah NIM)
 4. Hapus data di awal
 5. Hapus data di akhir
 6. Tampilkan data
 7. Tampilkan data secara terbalik
 0. Keluar
- Pilih menu : 6
 NIM : 123005
 Nama : Harry
 Kelas : 1A
 IPK : 3.76

- Pertanyaan

1. Perhatikan potongan kode berikut pada method removeFirst():

```
head = head.next;
head.prev = null;
```

Jelaskan fungsi masing-masing statement tersebut pada proses penghapusan node!

2. Modifikasi method removeFirst() dan removeLast() agar program menampilkan data yang berhasil dihapus!

Jawaban

1. head = head.next; untuk memindahkan pointer head ke node kedua, sehingga node pertama tidak lagi terhubung sebagai head. Sedangkan untuk head.prev = null; itu untuk memutuskan hubungan prev dari head baru ke node lama, memastikan node pertama benar-benar terlepas dari linked list.

2. Modifikasi :

```
public void removeFirst() {
    if (isEmpty()) {
        System.out.println("Linked List kosong.");
        return;
    }
```

```
    System.out.println("Data yang dihapus:");
    head.data.tampil();
```

```
    if (head == tail) {
        head = tail = null;
    } else {
        head = head.next;
        head.prev = null;
    }
```

```
    }  
}  
  
public void removeLast() {  
    if (isEmpty()) {  
        System.out.println("Linked List kosong.");  
        return;  
    }  
  
    System.out.println("Data yang dihapus:");  
    tail.data.tampil();  
  
    if (head == tail) {  
        head = tail = null;  
    } else {  
        tail = tail.prev;  
        tail.next = null;  
    }  
}
```